



Kisi-Kisi LKS
Web Technologies
Kabupaten Kudus
Tahun 2026

CLIENT SIDE MODULE

CONTENTS

This module has the following files:

1. MODULE_CLIENT_SIDE.docx
2. MODULE_CLIENT_SIDE.pdf
3. MODULE_CLIENT_SIDE_MEDIA.zip

INTRODUCTION

You are asked to develop a game called **World Head Football** using HTML and CSS and develop client-side programming using JavaScript. Some media files are available to you in a zip file. You can create more media and modify anything in the media if you want. Your game needs to be developed in a tablet resolution (1000 x 600 pixels). In bigger resolution, the game must be centered in the screen both horizontally and vertically.

DESCRIPTION OF PROJECTS AND TASKS

This is a module of 3 hours. Your first 1 hours must be used to create the initial layout using HTML/CSS. Your layout should follow the design that you created. The final 2 hours you will create the functionality of the game using JavaScript that allows the game to work correctly in different web browsers.

World Head Football game screen should have meet these requirements below:

1. Player Name
2. Gameboard
3. Player Character
4. Country Flag
5. Total Score
6. Timer
7. Match History

Design and Initial Layout

1. **Develop the initial markup (HTML + CSS) of your game application.** Overall screen must be within 1000 x 600 pixels and centered on the screen.
2. **The design should be delivered in dark mode color.** You are free to choose dark color as long as it has the user convenience.
3. **You are free to decorate** the game screen design as long as it meets the requirements.
4. **The HTML and CSS** code must be valid in the W3C standards for HTML5 and CSS3 rules in accordance with the WCAG and standard ARIA (Accessible Rich Internet Applications Suite)

Game Functionalities

1. **Show game welcome** in the center after pages are loaded.
2. **Players can go to the game** after filling the username field and click the **“Play Game”** button at the bottom of the welcome page.

3. **The “Play Game” button should be disabled** if the user did not input the username.
4. User must choose one from each option:
 - . one of three levels (easy, medium, hard).
 - . one of two balls to be used.
 - . one of many countries as player 1 and one as player 2.
2. **Game instructions should be shown** after the page is loaded.
3. **After clicking the “Instruction” button**, it shows game instructions.
4. **Users can close instructions** after clicking the “X” button.
5. **Show countdown for three seconds in the center of screen** after the user clicked the play button before the game started playing.
6. **When the game starts**, the timer will start with time according to level.
 - . ‘30 seconds’ for easy level
 - . ‘20 seconds’ for medium level
 - . ‘15 seconds’ for hard level
2. **The player 1 position** will be on the left and the player 2 will be on the right
3. **The ball will appear** from top to bottom in the middle of 2 characters.
4. **The ball will bounce** when it hits the body part of the character.
5. **Player 1 move the character** using the following buttons:
 - a. “A” to move left
 - b. “W” to jump
 - c. “D” to move right
 - d. “Space” to kick the ball
6. **Player 2 move the character** using the following buttons:
 - a. “Left arrow” to move left
 - b. “Right arrow” to move right
 - c. “Up arrow” to jump
 - d. “Enter” to kick the ball
7. **Items will drop** every 5 seconds.
8. **The items that will drop** are as follows:
 - a. Increase Ball box to increase ball size
 - b. Decrease Ball box to decrease ball size
 - c. Diamond ice to freeze ball in 3 seconds
9. **Item will disappears** after being hit by the ball
10. **Walking animations are showing** as the characters are moving.
11. **The score will be increased** if the player can score a goal against the opponent.
12. **After a goal**, the ball will disappear and reappear from top to bottom in the middle of 2 characters.
13. **Players can pause** the game.
14. Press **Esc** to open the **pause popup**. The game should be in a paused state when opening the popup.
15. Press **Esc** again to **continue** or click the “**continue**” button and the countdown should appear before the game continues.
16. **Game Over** when the timer time is up.
17. **If the time is over but the score is still the same**, then one of the characters must score again, then the game is over.
18. **Show popup after game over** to display the player username, the countries, scores, save match history button, and restart button.

19. **Match results should be saved in the local storage** after the player clicks the “**Save Score**” button.
20. **Players are able to** see the match history after clicking the Match History Button.
21. **Match history can be sorted** by score or last matches based on user choices.
22. **The game needs to work correctly** in Google Chrome.

EXAMPLE

These following images are for example purposes only. You may design your own game layout.



INSTRUCTION FOR COMPETITORS

1. The media files are available in the ZIP file. You can modify the supplied files and create new media files to ensure the correct functionality and improve the application.
2. The entry file should be '**XX_CLIENT_SIDE/index.html**'
3. You should create additional images for each of the requested resolutions to highlight hidden elements, animations, interactions, or any additional information that will assist in the presentation of the game design.
4. Save the working game to the directory on the server named "**XX_CLIENT_SIDE**". Be sure that your main file is called **index.html**.

SERVER SIDE MODULE

CONTENTS

This module has the following files:

1. MODULE_SERVER_SIDE.docx
2. MODULE_SERVER_SIDE.pdf
3. MODULE_SERVER_SIDE_MEDIA.zip

INTRODUCTION

In this module, you are asked to create a social media website called “Facegram” where in the website a user can see other user's posts by following the user. Users can also register as a private account so other users who are not following could not see posts. The detailed description and tools that you can use will be described below.

DESCRIPTION OF PROJECT AND TASKS

This module is divided into 2 phases. In the first phase, you will create a REST Api then in the second phase you will create a frontend application with the following provided frameworks:

- Laravel (latest)
- React (latest with react-router and axios)
- Vue (latest with vue-router and axios)

Phase 1 - REST API

For all endpoints, the request header as default sets Content-type: application/json, but some endpoints have specified Content-type header so you have to adjust it.

1. Authentication

You can use **sanctum** for the token management and it must be valid when placed in the Authorization request header as a **Bearer token**.

. Register

Endpoint	/api/v1/auth/register
Method	POST
Description	For user to register account
Request	Body: full_name (required) bio (required, max 100 chars) username (required, min 3 chars, unique, only alphanumeric, dot “.” or underscore “_” allowed) password (required, min 6 chars) is_private (boolean)
Response Success	HTTP Status Code: 201 Body:

	<pre>{ "message": "Register success", "token": "7 xuPEGo7jtV8lhoE2Mp3am1rwtlpYUTewXBTcH78Af922f116", "user": { "full_name": "Budi Budiman", "bio": "stop dreaming, start doing", "username": "budi.budiman", "is_private": true, "id": 101 } }</pre>
Response Invalid Field	HTTP Status Code: 422 Body: <pre>{ "message": "Invalid field", "errors": { "full_name": ["The full name field is required."], "username": ["The username has already been taken."], "password": ["The password field must be at least 6 characters."], "bio": ["The bio field is required."] } }</pre>

b. Login

Endpoint	/api/v1/auth/login
Method	POST
Description	For user to log into system
Request	Body: <pre>username password</pre>
Response Success	HTTP Status Code: 200 Body: <pre>{</pre>

	<pre> "message": "Login success", "token": "2 Qnt2aqHli0EVwCz3rSH0yiFIKMqey38SdkUZntRvf0e507ff", "user": { "id": 1, "full_name": "John Doe", "username": "john.doe", "bio": "The best way to predict the future is to create it.", "is_private": 0, "created_at": "2023-10-14T15:04:33.000000Z" } </pre>
Response Failed	HTTP Status Code: 401 Body: <pre> { "message": "Wrong username or password" } </pre>

c. Logout

Endpoint	/api/v1/auth/logout
Method	POST
Description	For user to logout
Request	Headers: Authorization: Bearer <token> Body: username password
Response Success	HTTP Status Code: 200 Body: <pre> { "message": "Logout success" } </pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre> { "message": "Unauthenticated." } </pre>

0. Post

. Create new post

Endpoint	/api/v1/posts
Method	POST
Description	For user to create a post
Request	Headers: Authorization: Bearer <token> Content-type: multipart/form-data Body: caption (required) attachments (required, array of image files jpg, jpeg, webp, png or gif)
Response Success	HTTP Status Code: 201 Body: <pre>{ "message": "Create post success" }</pre>
Response Invalid Field	Body: <pre>{ "message": "Invalid field", "errors": { "caption": ["The caption field is required."], "attachments.0": ["The attachments.0 field must be a file of type: png, jpg, jpeg, webp."] } }</pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre>{ "message": "Unauthenticated." }</pre>

b. Delete post

Endpoint	/api/v1/posts/:id
Method	DELETE
Description	For user to delete a post

Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 204
Response Post Not Found	HTTP Status Code: 404 Body: { "message": "Post not found" }
Response Try to delete another user's post	HTTP Status Code: 403 Body: { "message": "Forbidden access" }
Response Invalid Token	HTTP Status Code: 401 Body: { "message": "Unauthenticated." }

c. Get posts

Endpoint	/api/v1/posts
Method	GET
Description	For user to get all posts
Request	Headers: Authorization: Bearer <token> Params: page (integer, minimum 0 and default 0) size (integer, minimum 1 and default 10)
Response Success	HTTP Status Code: 200 Body: { "page": 0, "size": 10, "posts": [{ "id": 366, "caption": "Letting my soul wander", "created_at": "2023-07-26 00:34:10", "deleted_at": null,

	<pre> "user": { "id": 61, "full_name": "Irving Greenfelder", "username": "talía94", "bio": "Success is not final, failure is not fatal: It is the courage to continue that counts.", "is_private": 0, "created_at": "2023-10-18 19:16:53" }, "attachments": [{ "id": 750, "storage_path": "posts/652fc6284eb76.jpg" }]] } </pre>
Response Invalid Params	HTTP Status Code: 422 Body: <pre> { "message": "Invalid field", "errors": { "page": ["The page field must be at least 0."], "size": ["The size field must be a number."] } } </pre>
Response Try to delete another user's post	HTTP Status Code: 403 Body: <pre> { "message": "Forbidden access" } </pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre> { "message": "Unauthenticated." } </pre>

0. Following

. Follow a user

Endpoint	/api/v1/users/:username/follow
Method	POST
Description	For user to follow another user
Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 200 Body: { "message": "Follow success", "status": "following" "requested" }
Response User not found	HTTP Status Code: 404 Body: { "message": "User not found" }
Response Try to follow own user account	HTTP Status Code: 422 Body: { "message": "You are not allowed to follow yourself" }
Response Already Followed	HTTP Status Code: 422 Body: { "message": "You are already followed", "status": "following" "requested" }
Response Invalid Token	HTTP Status Code: 401 Body: { "message": "Unauthenticated." }

b. Unfollow a user

Endpoint	/api/v1/users/:username/unfollow
Method	DELETE
Description	For user to unfollow a followed user
Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 204
Response User not found	HTTP Status Code: 404 Body: { "message": "User not found" }
Response Not Following the User	HTTP Status Code: 422 Body: { "message": "You are not following the user" }
Response Invalid Token	HTTP Status Code: 401 Body: { "message": "Unauthenticated." }

c. Get following users

Endpoint	/api/v1/following
Method	GET
Description	For users to get their following users
Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 200 Body: <pre>{ "following": [{ "id": 91, "full_name": "Miss Ayana Russel", "username": "chartmann", "bio": "In a world where you can be anything, be kind.", "is_private": 0, "created_at": "2023-10-18 19:16:55", "is_requested": true false }, ] }</pre>
Response User not found	HTTP Status Code: 404 Body: <pre>{ "message": "User not found" }</pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre>{ "message": "Unauthenticated." }</pre>

0. Followers

- Accept follow request

Endpoint	/api/v1/users/:username/accept
Method	PUT
Description	For users to get their following users
Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 200 Body: { "message": "Follow request accepted" }
Response User not found	HTTP Status Code: 404 Body: { "message": "User not found" }
Response Already Accepted	HTTP Status Code: 422 Body: { "message": "Follow request is already accepted" }
Response User Not Following	HTTP Status Code: 422 Body: { "message": "The user is not following you" }
Response Invalid Token	HTTP Status Code: 401 Body: { "message": "Unauthenticated." }

b. Get follower users

Endpoint	/api/v1/users/:username/followers
Method	GET
Description	For users to get their following users
Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 200 Body: <pre>{ "followers": [{ "id": 91, "full_name": "Miss Ayana Russel", "username": "chartmann", "bio": "In a world where you can be anything, be kind.", "is_private": 0, "created_at": "2023-10-18 19:16:55", "is_requested": true false }, ] }</pre>
Response User not found	HTTP Status Code: 404 Body: <pre>{ "message": "User not found" }</pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre>{ "message": "Unauthenticated." }</pre>

0. User

- . Get users

Endpoint	/api/v1/users
Method	GET
Description	Get all users that are not followed by logged in users. Logged in users should be hidden in the list.
Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 200 Body: <pre>{ "users": [{ "id": 1, "full_name": "John Doe", "username": "john.doe", "bio": "The best way to predict the future is to create it.", "is_private": 0, "created_at": "2023-10-14T15:04:33.000000Z", "updated_at": "2023-10-14T15:04:33.000000Z" }, ] }</pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre>{ "message": "Unauthenticated." }</pre>

b. Get detail user

Endpoint	/api/v1/users/:username
Method	GET
Description	For users to get detailed user data. Field posts should be hidden if the user is a private user and the follow status is not following or is requested.
Request	Headers: Authorization: Bearer <token>
Response Success	HTTP Status Code: 200 Body: <pre>{ "id": 2, "full_name": "Richard Roe", "username": "richard.roe", "bio": "Leave a little sparkle wherever you go.", "is_private": 1, "created_at": "2023-10-18 19:16:50", "is_your_account": false, "following_status": "not-following" "requested" "following", "posts_count": 6, "followers_count": 20, "following_count": 18, "posts": [{ "id": 7, "caption": "Blissfully lost in the beauty of the moment.", "created_at": "2014-08-26 10:42:31", "deleted_at": null, "attachments": [{ "id": 13, "storage_path": "posts/shutterstock_1464930743-scaled.webp" }, ] }, ] }</pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre>{ "message": "Unauthenticated." }</pre>

Response User not found	HTTP Status Code: 404 Body: <pre>{ "message": "User not found" }</pre>
Response Invalid Token	HTTP Status Code: 401 Body: <pre>{ "message": "Unauthenticated." }</pre>

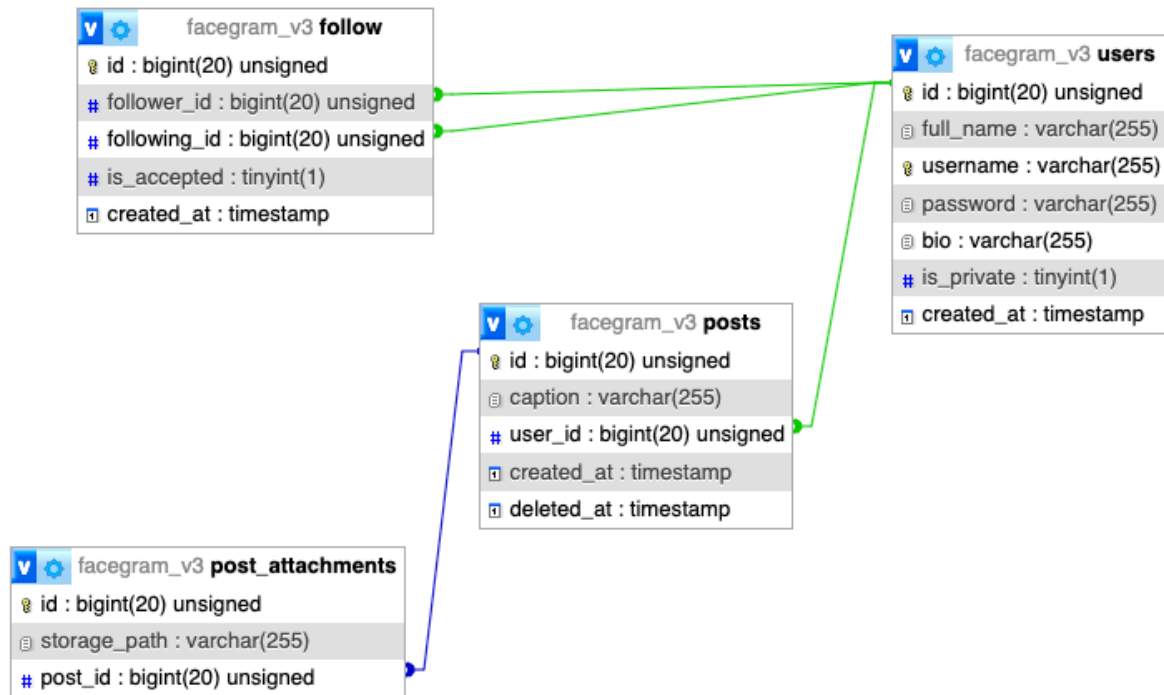
Phase 2 - Frontend Development

Template HTML is provided. You are required to use the template and you are recommended to modify the template design without affecting the functionality of the frontend.

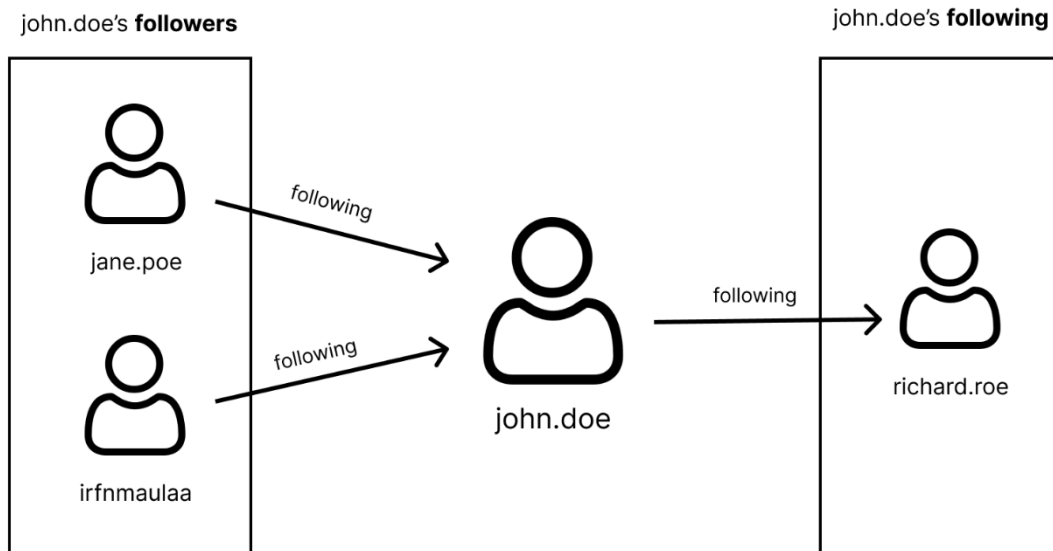
Page	Description Tasks
General	- Document title should be reflected to the current page - Display username and logout button on the navbar after login success - Clicking username will go to logged in user profile page - User can log out after clicking logout button on the navbar
Register	- If user logged in, the page will be redirected to own user profile page - User can register account - If register fails, display all of errors message - If logged in user try to access this page, the page should be redirected to own user profile page
Login	- If user logged in, the page will be redirected to own user profile page - User can login with existing credentials - If login fails, display all of errors message - If logged in user try to access this page, the page should be redirected to own user profile page
Homepage	- If user not logged in, the page will be redirected to login page News Feed section - Display following user's posts and own posts sorted by newest post on the newsfeed section - When page loads, display only first 10 newest posts - If the vertical scroll is stuck at the bottom, load the next 7 posts Explore people section - Display all users that are not followed by the logged in user - Clicking on username will go to that user profile page

	Follow requests section (for private account) • Display follow request users • The logged in user can accept/approve a follow request • If the logged in user is not a private account or there is no any follow request, hide this section
User profile	Page requirements: • If user not logged in, the page will be redirected to login page User information: • See profile (full_name, username, bio) • See posts list and count • See followers list and count • See following list and count Follow/unfollow: • User can follow another user • User can unfollow a followed user Posts section: • Display user's posts where the visited user is not a private account or following account • Users can delete their posts • Display "The account is private" if logged in user is not following the visited private user profile page
Create new post	• If user not logged in, the page will be redirected to login page • User can create new post with following fields: • caption (text, required) • attachments (file, can attach multiple files) • If create new post fails, display all of errors message

ER DIAGRAM



ADDITIONAL EXPLANATION



MEDIA FILES

- Frameworks (laravel, react and vue)
- SQL File (structure and records. allowed to modify)
- Postman collection
- Template HTML (based on bootstrap v5.3)

INSTRUCTION FOR COMPETITORS

- Database (**db-dump.sql**) and ER diagram (**db-diagram.pdf**) should be exported and saved under the root folder **<host>/XX_SERVER_MODULE**
- REST Api should be reached on **<host>/XX_SERVER_MODULE/BACKEND** where xx is PC number.
No port and no /public suffix path.
- Frontend app should be reached on **<host>/XX_SERVER_MODULE/FRONTEND** where xx is PC number.
No port allowed
- **Zip XX_SERVER_MODULE** and submit to the submission page. **Make sure to exclude the vendor and node_modules** folder when zipping files.